

● DYNAMOUS · LIVE WEBINAR

PRDs, AI Specs & Intent Engineering

Where the engineering decisions actually live —
and how you make them with your agent.

Rasmus presenting · Cole hosting · [Dynamous](#)

A PRD is intent.

An AI spec is engineering decisions.

AI didn't eliminate the engineering — it moved it upstream, into the spec.

Planning is the most leveraged step

WINS

Opus plans + Kimi builds

beats

LOSES

Kimi plans + Opus builds

The model you **plan** with > the model you build with.

Nail the plan and a smaller model can carry the build.

When agentic coding goes sideways, what's the real root cause?

The model?

The tools?

Or something **earlier**?

Drop your take in the chat...



What does “PRD” actually stand for?

First thing that comes to mind. No Googling.

• YOUR TURN

Drop it in the chat...



A PRD defines a **problem** and our **hypotheses** about solving it — in a form a team can challenge before building, and judge after shipping.

It captures a discovery **conclusion** — not the discovery itself.

Discovery now happens **with** the agent: feedback, opportunities, the competitive landscape.

PRD vs spec vs roadmap

PRD

The problem & the intent

what / why

SPEC

The engineering decisions

how

ROADMAP

The sequencing

when

Most confusion comes from collapsing all three into one document.

Five things that make a PRD good

- 01 Problem grounded in **evidence** — not “users want X”
- 02 **Hypothesis with a separate right and wrong condition** — the most-skipped line
- 03 Success metrics that are specific & **outcome-shaped** — not “engagement”
- 04 Explicit **non-goals**
- 05 Open questions **named, not hidden**

10%

Solving a real pain is only the first slice.

Users already cope today — a workaround, a competitor, or just tolerating it.

The bet: do you solve it **so much better they actually switch?**

“Solving the problem is table stakes. Solving it better than whatever they do today is the actual product.”

The hypothesis template

We believe [change] will cause [these users] to [do Y],
resulting in [outcome].

We'll know we're **right** if [leading signal] within [timeframe].

We'll know we're **wrong** if [counter-signal / guardrail moves].

The **wrong** condition is the one everyone leaves out — and the one that makes the bet falsifiable.

Which PRD is stronger?

Same feature — food delivery, win back lapsed customers.

VERSION A

Problem: lapsed users don't reorder

Why now: churn is up this quarter

Who: customers inactive 30+ days

Hypothesis: give them faster access to their favorites

VERSION B

Problem: re-entry friction after a lapse

Why now: 1st-month repeat rate falling

Who: lapsed 30+ days, ordered $\geq 2\times$ before

Hypothesis: lower the cost of restarting → measurable return within 14 days

Vote A or B in the chat — and say why...

⌘← vote



How many of you have written a PRD that **named the solution** in the first paragraph?

It's a habit to break, not a sin. Everyone's done it.

• REACT IN THE CHAT

...or just nod quietly



If only one solution fits, it's a spec in disguise

The test: if **only one** solution could fit your problem statement, you've written a spec.

A good problem leaves room for **more than one** answer.

outcomes over outputs (Cutler) · generate multiple solutions before choosing (Torres, OST)

Solution-prescriptive vs intent-framed

✗ PRESCRIPTIVE

"Add a reply button to every message."

✓ INTENT-FRAMED

"Past ~100 msgs/day, conversations collide and active users disengage — give them a way to group related replies so they stay."

The first kills **6 of Slack's 7 explorations** before anyone looks at the problem.

What the AI scene gets wrong

~~"A PRD is an instruction set"~~ → that's a spec wearing a PRD label

~~"The AI will figure out the how"~~ → agents can't infer from omission ("vague specs → vague code")

~~"Idea → product in 30 min"~~ → deletes the engineering judgment

The opening we own: PRD-as-hypothesis is missing from the conversation.

**Build the thinnest slice that tests
your hypothesis. Then measure.**

Holds → continue. Doesn't → throw it away and start over.

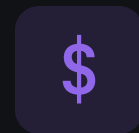
Problem, or solution?

- 1 New customers who place one order rarely come back for a second within their first month.
- 2 Add a one-tap “reorder your last meal” button to the home screen.
- 3 Customers need a quicker way to get back to the meals they've ordered before.
- 4 Customers abandon their cart because checkout has too many steps.

Vote **P** or **S** on each in the chat...



Three altitudes



Prompt engineering
optimizes language



Context engineering
optimizes what the model knows



Intent engineering
optimizes business impact

THE LEVERAGE

Most energy goes to the bottom two. The leverage is at the top.

"Code is a lossy projection from the specification."

— Sean Grove, OpenAI

We keep the code and throw away the spec —
"shred the source, version-control the binary."

Karpathy: **prompts are the new source code.**

The naming problem

“Requirement” implies **certainty**. A PRD carries **direction under uncertainty**.

KEEP THE REBRAND

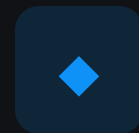
“Intent” is the better word.

REJECT THE PRACTICE

“Write an intent PRD, let the AI figure out the how” = delegation, not discipline.

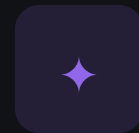
Real intent engineering is the **loop** — with you making the engineering calls.

Two altitudes, one collaborator



Intent altitude → the PRD

what / why — with AI



Engineering altitude → the spec

how — with AI



Agent

writes the code against the spec

The discipline: know which altitude you're at — don't let the agent quietly make the how-calls.

What a PRD must never decide

`Library & version` React 18 + Vite + Tailwind, not “a React project”

`Data model` relationships between entities

`Security` boundaries — “**never commit secrets**”

`Testing` architecture

`Error handling` & retries

`Project structure`

Top finding across 2,500 repos: “**never commit secrets.**” These are spec decisions — not PRD lines.

An AI spec for Slack threads

DECISION	WHY IT'S ENGINEERING
<code>Data model</code> parent_message_id vs thread_replies table	read perf, migration risk
<code>API</code> embed metadata vs GET /threads/:id	request volume, latency
<code>Mobile sync</code> incremental vs full re-sync	data, battery, offline
<code>Notifications</code> who auto-follows on @-mention	notification noise
<code>Search</code> indexed under parent or standalone	storage, relevance

None of these are in Slack's public article. They were **made** — just not published.

170

of **1,645** audited Lovable apps
leaked user data. CVE-2025-
48757

RLS, secrets, webhook verification — never in the pasted “PRD.”

Skipped engineering decisions don't vanish. They become **vulnerabilities**.

```
Klarna = org-level. Lovable = build-level. “Demo is works.any();  
product is works.all().”
```


Vibe coding vs vibe engineering

VIBE CODING

Fast, loose, no spec, no review. Fine for throwaway — fatal at org scale. (Klarna.)

VIBE ENGINEERING

Experts using AI to amplify expertise that lives **in the spec**.

The engineering didn't disappear. It moved upstream — which makes the PRD matter more.

An MVP is the **thinnest line** you can build to prove — end to end — that your hypothesis is right or wrong.

Holds → write the full spec, build it proper.

Doesn't → throw away a slice, not six months.

Not “build the product.” Build the thinnest thing that proves the hypothesis.

Who are you building for?

FOR YOURSELF

You're the user. Prove or kill it on yourself, fast. You can jump closer to the solution.

FOR SOMEONE ELSE

You can't introspect their needs. **Building it right > building it.**

Thinnest MVP + experiments = how you find out what “right” actually is.

Cagan's four risks

Value

do they want it? → interviews / fake doors

Usability

can they use it? → prototype + watch

Feasibility

can we build it? → engineering spike

Viability

does it work for the business? → stakeholder review

Most teams over-invest **feasibility**, under-invest **value** — and value means wanting it [more than the alternative](#).

Slack vs the four risks

Value	Assumed — under-tested
Usability	The killer. Exploration 4 looked great until dogfooding showed it made channels harder to read.
Feasibility	Mostly solved — not the bottleneck
Viability	Fixed by hiding replies (Expl. 5) — “the single most meaningful change”

Spike it, or build it?

TWO-WAY DOOR

Reversible → just build it.

ONE-WAY DOOR

Expensive to undo → spike first.

Al drops the “just build it” bar — **but only with pre-committed success criteria.**

Cheap builds → run several hypothesis tests [in parallel](#), each with its own right/wrong.

Hypothesis → spike

We believe [change] → [users] → [do Y] → [outcome].

To verify: [method] with [sample] over [timebox]. ← the one added line

Right if [signal] / wrong if [counter-signal].

Hypothesis lives in the PRD. Verification lives in the spike doc.

Spike it, or just build it?

- 1 Ship a “Reorder your usual” card on the home screen, behind a 5% feature flag.
Cheap, isolated, and trivial to roll back.
- 2 Launch group ordering — which means migrating a live orders table used by millions of customers.
A one-way migration that's expensive and risky to undo.
- 3 Put AI meal recommendations on the home screen.
Cheap to build — but you're unsure anyone wants it, and it changes the home screen for every user.

Vote `spike` or `build` in the chat...



When is it overplanning?

It depends on **what** you're building and **who** for.

Just you, brand new, you know what you want → **a page is enough.**

A feature real users depend on → **30 hours is reasonable.**

The real question isn't "how much planning?" — it's "**what's the cost of being wrong?**"

"Overplanning costs you hours. Underplanning costs you your reputation."

Monday morning

PM

Find where your PRD named a solution → rewrite as a problem + add the wrong-condition.

ENGINEER

Before you prompt, write the six spec decisions: stack, data model, security, testing, errors, structure.

BOTH

Notice which altitude you're at — don't let the agent blur them.

“Failing is an essential part of the design process, and you just have to be okay with it.”

– Slack Design

Don't be Klarna: building fast with no wrong-condition is how you reverse on stage.

A course goes deeper on this — soon.

Questions?